Data Processing → Model Training SOP

> Run `conda activate AR` to activate the environment before running any script in this document.

Overview

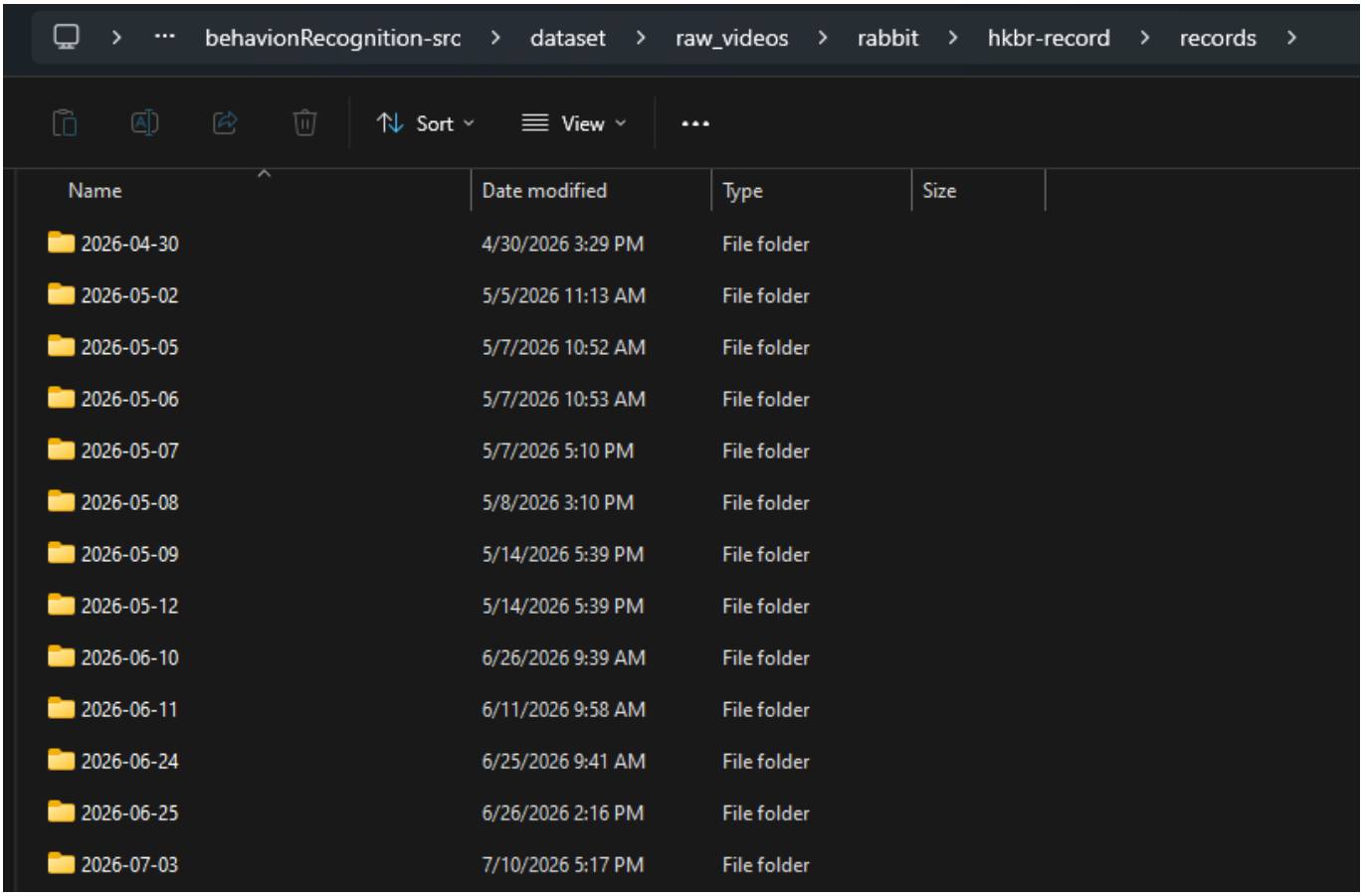
```
Raw videos → cut into single-action clips → data processing script (YOLO crop + slicing)
→ merge with historical data into a new snapshot → point config.yaml at the new snapshot
→ teacher fine-tuning → student distillation
```

Step 1: Collect raw videos

From a deployed site (e.g. logging into HKBR via AnyDesk), download the recorded raw long videos to the local training machine, and archive them by site/date:

```
dataset/raw_videos/rabbit/<site>/records/<date>        # rabbit
dataset/raw_videos/pet/normal/<site>/records/<date>    # cat/dog
```

At this point the videos are full-length recordings — a single video may contain multiple behaviors, **so they cannot be used for training directly**.



Step 2: Manual clipping and classification

Manually clip and classify the videos collected in Step 1.

The only rule: one video clip must correspond to exactly one clear behavior. The folder names must strictly match: cat/dog category folders are `[eat, drink, other]`, rabbit category folders are `[resting, active, eating, drinking]`.

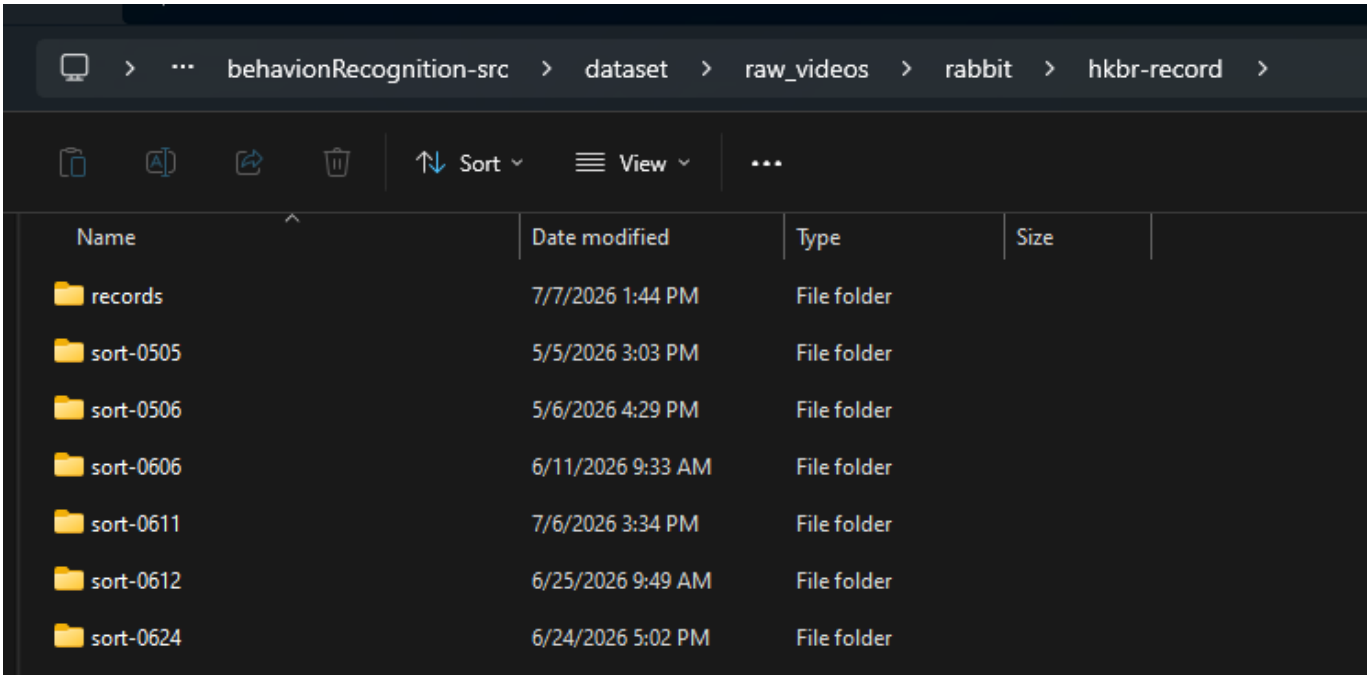
- Cat/dog categories: `eat` / `drink` / `other`
- Rabbit categories: `resting` / `active` / `eating` / `drinking`

After clipping, put the clips into the matching category folder (the folder name must contain the category keyword), e.g.:

```
sorted-0626/
  eat/
    clip_001.mp4
  drink/
    clip_002.mp4
  other/
    clip_003.mp4
```

Step 3: Sort data by date

Put the organized clips into the corresponding `sorted-` subdirectory under `dataset/raw_videos/<site>/` (e.g. `dataset/raw_videos/rabbit/hkbr-record/sorted-0624/`).



Step 4: Run the data processing script to produce training samples

Script	Output format
`utils/abnormal_crop.py`	One folder per sample: per-frame JPGs (16 frames) + `label.txt`
`utils/abnormal_crop_video.py`	One cropped mp4 clip per sample, stored under category folders

Both scripts share the same logic (YOLO detects and crops the target region → slices into fixed-length clips → outputs into a new "date snapshot" folder). The only difference is **the physical output format** — either format can be used directly by the training scripts. Going forward, prefer `utils/abnormal_crop_video.py` and train directly on video clips.

Operating steps (identical for both scripts):

1. Open the script and find the "config parameters" section at the top.

2. `INPUT_BASE_DIRS` : change to the raw-clip directories to process this run (the paths from Step 3); multiple paths can be listed.

3. `OUTPUT_BASE_DIR` : **change to a new date-snapshot directory**, following the "date" naming convention — e.g. change `dataset/rabbit/0626-videos` to `dataset/rabbit/0703-videos` (use today's date). **Do not overwrite an existing snapshot directory.**

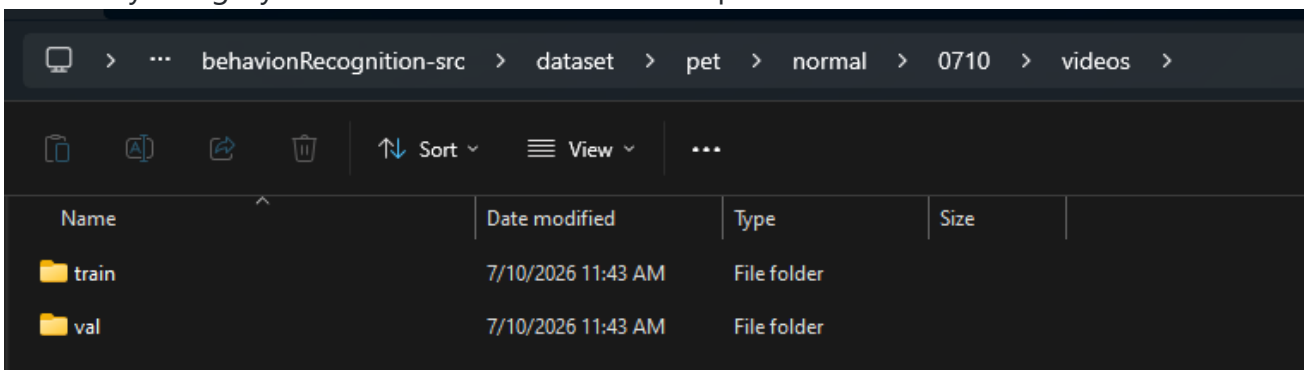
4. Confirm `CLASS_NAMES` (and `LABEL_MAP` in `abnormal_crop.py`) match the category names used for clipping in Step 2, and that `YOLO_MODEL_PATH` is the detector weights for the corresponding species.

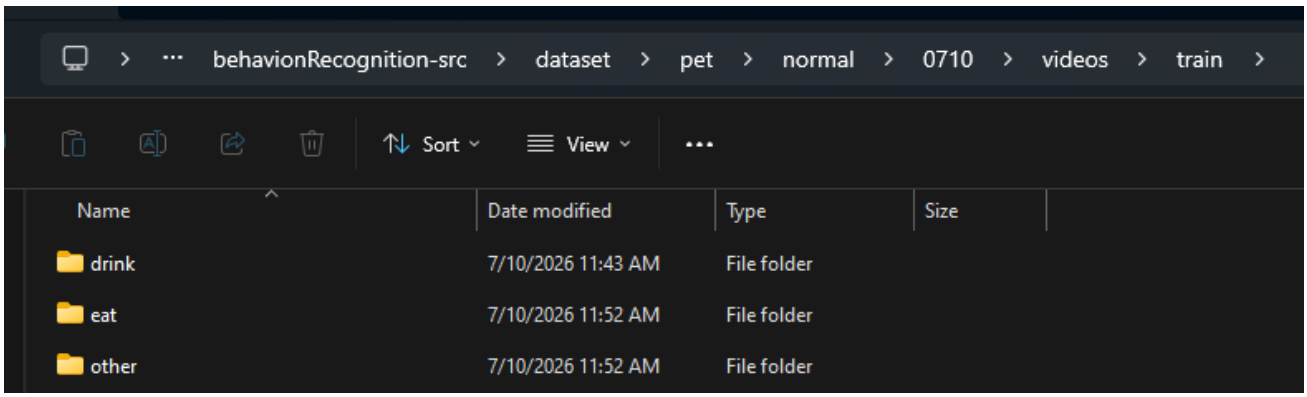
```
INPUT_BASE_DIRS = [
    r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sorted-0709",
    r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sorted-0710",
    # r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sort\0430",
    # r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sort\0501",
    # r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sort\0503",
    # r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sort\0504",
    # # r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sort\0626",
    # r"E:\ActionRecognition\behavionRecognition-src\dataset\raw_videos\pet\normal\sort\0629",
]
# CLASS_NAMES     = [
#                 "eating",
#                 "active",
#                 "resting",
#                 "drinking",
#                 "moving"
#                    ]     # rabbit dataset class names

CLASS_NAMES     = ["eat", "drink", "other"] # dog-cat dataset class names
OUTPUT_BASE_DIR  = r"E:\ActionRecognition\behavionRecognition-src\dataset\pet\normal\0710\videos"
YOLO_MODEL_PATH  = r"E:\ActionRecognition\detection-src\cat_dog_detect\runs\yolo11n_scale0.6_fliplr0.4\weights\best.pt" # dog-cat detect model
# YOLO_MODEL_PATH = r'e:\ActionRecognition\detection-src\rabbit-detect\runs\checkpoints\yolo11n.pt3\weights\rabbit-yolo11n-0603.pt' # rabbit detect model
TARGET_CLASSES   = [1, 2]   # dog-cat labels in the model, 1: cat, 2: dog
# TARGET_CLASSES   = [0]    # rabbit label in the model, 0: rabbit
```

5. Run the script directly

6. After it finishes, check that the output directory has `train/` and `val/` subdirectories, and that every category has a non-zero number of samples.





Step 5: Merge old and new data into a full snapshot

Key action: the rule for `dataset/.../<date>` folders is that each date folder represents "the complete training data as of this date," not "only the data added that day." So the new snapshot produced in Step 4 needs the historical samples from the previous snapshot copied in, to merge into one complete version:

1. Find the current "latest valid" snapshot directory (the folder with the most recent date for the same species).
2. Copy all sample folders/files under the old snapshot's `train/` and `val/` into the corresponding `train/` / `val/` of the new snapshot (filenames carry date and source info, so name collisions are unlikely — a plain merge copy is fine).
3. Once merged, the new snapshot directory is "the current valid data." The old date snapshot is kept as historical record — **do not add new data to it afterward**.

Step 6: Update config.yaml to point at the new data

Open the root `config.yaml` and edit the `task_configs.normal` block (lines 8-16). Cat/dog and rabbit share the same config slot — point it at the latest snapshot for whichever species you're training:

```yaml
task_configs:
  normal:
    dataset_path: "dataset/pet/normal/0710/videos"      # change to the new snapshot
path produced in Step 5
    class_names: ['eat','drink','other']   # confirm the category list and order match
    finetune:
      checkpoint_path: None
      output_dir: "outputs/pet/{date}/teacher_finetuned_{date}"
      output_model_name: "pet_teacher_{date}_ema-videos.pth"
```

⚠ **When switching between cat/dog and rabbit, the `pet` / `rabbit` substrings in all three fields — `dataset_path`, `output_dir`, `output_model_name` — must be updated together, not just `class_names`:**

- `dataset_path`: point to the latest snapshot for the target species; whether the path says `pet` or `rabbit` must match the species (e.g. `dataset/pet/normal/0710/videos` vs `dataset/rabbit/0703-videos`).
- `output_dir` / `output_model_name`: likewise change the `pet` / `rabbit` substring in the path/filename to match the species, to avoid a cat/dog-trained teacher checkpoint ending up saved into the rabbit directory (or vice versa).

If these three fields aren't updated together, training will still run, but the resulting model will be saved under the wrong species directory and get mixed up with historical models.

```
8   task_configs:
9     normal:
10      dataset_path: r"E:\ActionRecognition\behavionRecognition-src\dataset\pet\normal\0710\videos"
11      class_names:  ['eat','drink','other'] # dog-cat: ['eat','drink','other'] rabbit: ["resting", "active", "eating", "drinking"]
12
13      finetune:
14        checkpoint_path: None
15        output_dir: "outputs/pet/{date}/teacher_finetuned_{date}"
16        output_model_name: "pet_teacher_{date}_ema-videos.pth"
```

Step 7: Teacher model fine-tuning

```
conda activate AR
python train/train_finetune_normal_3.py
```

Just run the script directly — the teacher model will be saved automatically to the corresponding date path: `outputs/pet/{date}/teacher_finetuned_{date}` or `outputs/rabbit/{date}/teacher_finetuned_{date}`.

Step 8: Student model distillation

Before running distillation, confirm that `config.yaml` line 111, `distillation.teacher_checkpoint`, points to the teacher checkpoint path just produced in Step 7 (otherwise distillation will use the old teacher, effectively not using the newly trained one).

```
106   distillation:
107     enabled: true
108     mode_to_use: "normal"
109     teacher_model: "repvit_m1"
110     student_backbone: "repvit_m1"
111     teacher_checkpoint: r"outputs\pet\0710\teacher_finetuned_3cls_0710\pet_teacher_0710_ema-videos.pth"
```

```
conda activate AR
python distill/distill_normal_3.py
```

- Also takes no arguments — it reads `config.yaml` automatically.
- After training completes, check whether `config.yaml`'s `task_configs.normal.distillation.output_dir` now contains two checkpoint files: `output_model_name` (raw student weights) and the corresponding `*_ema.pth` (EMA weights).

Step 9: Acceptance & delivery

- Confirm the checkpoint files exist in the Step 7 and Step 8 output directories, and that the validation accuracy in the training logs is a sane value (not 0 or near random-guess level).
- **Deployment uses the student model's EMA weights** (the `*_ema.pth` produced in Step 8) — use this file to update the deployed system.

Full run-through checklist

- [ ] Raw videos have been downloaded and archived to `dataset/raw_videos/...`
- [ ] Clipping and classification are complete, organized into category folders
- [ ] `abnormal_crop.py` or `abnormal_crop_video.py` has been run (either output format), producing a new date snapshot with samples under `train/val`
- [ ] The old snapshot's data has been merged into the new snapshot
- [ ] `config.yaml`'s `dataset_path` / `class_names` point at the new snapshot, and the `pet` / `rabbit` substrings in `output_dir` / `output_model_name` have been changed to match the species
- [ ] `train/train_finetune_normal_3.py` has finished running, teacher weights have been produced
- [ ] `config.yaml`'s `distillation.teacher_checkpoint` points at the new teacher weights
- [ ] `distill/distill_normal_3.py` has finished running, student weights + EMA weights have been produced
- [ ] The EMA weights have been used to update the deployed system